INTRODUCTION TO PANDAS

Programming for Data Science

Introduction

- Pandas is a Python library used for working with data sets.
- It has supports for analyzing, cleaning, exploring, and manipulating data.
- The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis“.
- Pandas allows us to analyze big data and make conclusions based on statistical theories.
- Pandas can clean messy data sets, and make them readable and relevant.

Using Pandas

C:\Users\Your Name>pip install **pandas**

```
# check pandas version
import pandas
print(pandas.__version__)

# alternative of above code
import pandas as pd
print(pd.__version__)
```

Pandas Series

- A **Pandas Series** is like a column in a table.
- It is a one-dimensional array holding data of any type.

Pandas Series

```
import pandas as pd

a = [10, 70, 20] # list
b = (10, 70, 20) # tuple

myvar1 = pd.Series(a)
myvar2 = pd.Series(b, index = ["x", "y", "z"])

print(myvar1)
print(myvar1[0], myvar1[2]) # cannot use negative index
print(myvar2)
print(myvar2["x"], myvar2["z"])
```

Pandas Series

```
import pandas as pd

a = [10, 70, 20] # list
b = (10, 70, 20) # tuple

myvar1 = pd.Series(a)
myvar2 = pd.Series(b, index = ["x", "y", "z"])

print(myvar1)
print(myvar1[0], myvar1[2]) # cannot use negative index
print(myvar2)
print(myvar2["x"], myvar2["z"])
```

A label can be used to access a specified value. If no label is specified, then values are labelled with index number starting from 0.

Pandas Series

```
import pandas as pd

a = [10, 70, 20] # list
b = (10, 70, 20) # tuple

myvar1 = pd.Series(a)
myvar2 = pd.Series(b, index = ["x", "y", "z"])

print(myvar1)
print(myvar1[0], myvar1[2]) # cannot use negative index
print(myvar2)
print(myvar2["x"], myvar2["z"])
```

Output:

```
0    10
1    70
2    20
dtype: int64
10 20
x    10
y    70
z    20
dtype: int64
10 20
```

Pandas Series

```
import pandas as pd

# dictionary
calories = {"day1": 420, "day2": 380, "day3": 390}

myvar3 = pd.Series(calories)
myvar4 = pd.Series(calories, index = ["day1", "day2"])

print(myvar3)
print(myvar3["day1"], myvar3["day3"])
print(myvar4)
print(myvar3["day1"], myvar3["day2"])
```

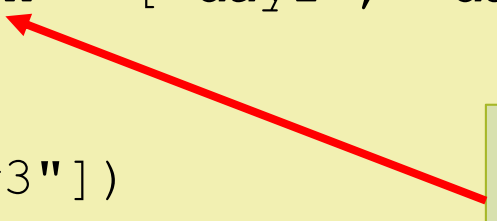
Pandas Series

```
import pandas as pd

# dictionary
calories = {"day1": 420, "day2": 380, "day3": 390}

myvar3 = pd.Series(calories)
myvar4 = pd.Series(calories, index = ["day1", "day2"])

print(myvar3)
print(myvar3["day1"], myvar3["day3"])
print(myvar4)
print(myvar3["day1"], myvar3["day2"])
```



Only specified items will be included in the series.

Pandas Series

```
import pandas as pd

# dictionary
calories = {"day1": 420, "day2": 380, "day3": 390}

myvar3 = pd.Series(calories)
myvar4 = pd.Series(calories, index = ["day1", "day2"])

print(myvar3)
print(myvar3["day1"], myvar3["day3"])
print(myvar4)
print(myvar3["day1"], myvar3["day2"])
```

Output:

```
day1    420
day2    380
day3    390
dtype: int64
420 390
day1    420
day2    380
dtype: int64
420 380
```

Pandas DataFrames

- Data sets in Pandas are usually multi-dimensional tables, called **DataFrames**.
- Series is like a column, a DataFrame is the whole table.
- A **Pandas DataFrame** is a 2-dimensional data structure, like a 2-dimensional array, or a table with rows and columns.

DataFrames

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

# create a DataFrame object from dictionary
df = pd.DataFrame(data)

print(df)
```

DataFrames

- Dictionary Keys will be column title and Dictionary Values will be inserted as the column values.
- Rows of the DataFram can be accessed using index starting from 0.

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

# create a DataFrame object from dictionary
df = pd.DataFrame(data)

print(df)
```

DataFrames

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

# create a DataFrame object from dictionary
df = pd.DataFrame(data)

print(df)
```

Output:

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

DataFrames

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

#load data into a DataFrame object:
df = pd.DataFrame(data)

print(df)
#use a row index:
print(df.loc[0])
#use a list of indexes:
print(df.loc[[0, 2]])
```

A DataFrame is like a table with rows and columns. Pandas use the *loc* attribute to return one or more specified row(s). It returns a Pandas Series.

DataFrames

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

#load data into a DataFrame object:
df = pd.DataFrame(data)

print(df)
#use a row index
print(df.loc[0])
#use a list of indexes
print(df.loc[[0, 2]])
```

Output:

```
   calories  duration
0       420        50
1       380        40
2       390        45
calories    420
duration     50
Name: 0, dtype: int64
   calories  duration
0       420        50
2       390        45
```

DataFrames

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)
#use a named index
print(df.loc["day2"])
#use a list of named indexes
print(df.loc[["day2", "day1"]])
```

With the index argument, we can name the indexes. Later we can use the named index in the *loc* attribute to return the specified row(s).

DataFrames

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1",

print(df)
#use a named index
print(df.loc["day2"])
#use a list of named indexes
print(df.loc[["day2", "day1"]])
```

Output:

```
       calories  duration
day1        420        50
day2        380        40
day3        390        45
calories     380
duration      40
Name: day2, dtype: int64
       calories  duration
day2        380        40
day1        420        50
```

Read CSV

```
import pandas as pd

df = pd.read_csv('data.csv’)

print(df)
print(df.to_string())
```

Read CSV

```
import pandas as pd

df = pd.read_csv('data.csv')

Print(df)
print(df.to_string())
```

If you have a large DataFrame with many rows, Pandas will only return the first 5 rows, and the last 5 rows.

`to_string()` prints the entire DataFrame

Read CSV

```
import pandas as pd

df = pd.read_csv('data.csv')

Print(df)
print(df.to_string())
```

Output:

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
..	...	...	...	...
164	60	105	140	290.8
165	60	110	145	300.0
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

Read CSV

- The number of rows returned is defined in Pandas option settings.
- We can check your system's maximum rows with the pd.options.display.max_rows statement.
- In my system the number is 60, which means that if the DataFrame contains more than 60 rows, the `print(df)` statement will return only the headers and the first and last 5 rows.
- You can change the maximum rows number with the same statement.

Read CSV

```
import pandas as pd

print(pd.options.display.max_rows)
pd.options.display.max_rows = 9999

df = pd.read_csv('data.csv')

print(df)
```

Read CSV

```
import pandas as pd

print(pd.options.display.max_rows)
pd.options.display.max_rows = 9999

df = pd.read_csv('data.csv')

print(df)
```

Now the entire DataFrame will be displayed.

Read JSON

- `read_json()` loads a JSON file into a DataFrame.
- JSON = Python Dictionary; JSON objects have the same format as Python dictionaries.
- If a JSON code is not in a file, but in a Python Dictionary, then it can be loaded into a DataFrame directly.

```
import pandas as pd

df = pd.read_json('data.json')

print(df.to_string())
```

Analyzing DataFrames

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head())     # first 5 rows
print(df.tail())     # last 5 rows
print(df.head(10))   # first 10 rows
print(df.tail(10))   # last 10 rows
```

Analyzing DataFrames

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.info())
```

Analyzing DataFrames

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.info())
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Duration  169 non-null    int64
 1   Pulse     169 non-null    int64
 2   Maxpulse  169 non-null    int64
 3   Calories  164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

Analyzing DataFrames

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.info())
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Duration  169 non-null    int64
 1   Pulse     169 non-null    int64
 2   Maxpulse  169 non-null    int64
 3   Calories  164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

Analyzing DataFrames

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.info())
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Duration  169 non-null    int64
 1   Pulse     169 non-null    int64
 2   Maxpulse  169 non-null    int64
 3   Calories  164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

Analyzing DataFrames

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.info())
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Duration  169 non-null    int64
 1   Pulse     169 non-null    int64
 2   Maxpulse  169 non-null    int64
 3   Calories  164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

Analyzing DataFrames

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.info())
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Duration  169 non-null    int64
 1   Pulse     169 non-null    int64
 2   Maxpulse  169 non-null    int64
 3   Calories  164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

Analyzing DataFrames

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.info())
```

Empty values, or Null values, can be bad when analyzing data, and we should consider handling them using different techniques. This is a step towards what is called **cleaning data**.

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Duration  169 non-null    int64
 1   Pulse     169 non-null    int64
 2   Maxpulse  169 non-null    int64
 3   Calories  164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

Data Correlations

- A great aspect of the Pandas module is the `corr()` method.
- The `corr()` method calculates the relationship between each column in your data set.

Useful PANDAS functions

- .read_csv() # loads a .csv file

Accessing/processing:

- df["column_name"]
  - .max(), .min(),
  - .idxmax(), .idxmin()
- <dataframe> <conditional>
- .loc[] – label-based accessing
- .iloc[] – index-based accessing
- .sort_values()
- .isnull(), .notnull()
- .dropna()
- .any()
- .values() E.g., df['column'].values()
  - (df['name'] == "Chris").any()
- [0:3] *# grab the first 3 rows of the DataFrame*

Grouping/Splitting/Aggregating:

- .groupby(),
- .get_groups()
- .drop()
- .merge(), .concat() .aggregate()
- .append()
- .sum() .median() .mean()

High-level viewing:

- .head() – first N observations
- .tail() – last N observations
- .describe() – statistics of the quantitative data
- .dtypes – the data types of the columns
- .columns – names of the columns
- .shape – the # of (rows, columns)

Useful PANDAS functions

- .read_csv() # loads a .csv file

Accessing/processing:

- df[“column_name”]
  - .max(), .min(),
  - .idxmax(), .idxmin()
- <dataframe> <conditional>
- .loc[] – label-based accessing
- .iloc[] – index-based accessing
- .sort_values()
- .isnull(), .notnull()
- .dropna()
- .any()
- .values() E.g., df[‘column’].values()
  - (df[‘name’] == “Chris”).any()
- [0:3] *# grab the first 3 rows of the DataFrame*

Grouping/Splitting/Aggregating:

```
df.loc[(df['Height'] >= 172)]
```

```
df.loc[(df['Height'] >= 172) & (df['Height'] <= 240)]
```

```
df.loc[(df['Height'] < 150) | (df['Height'] > 180)]
```

- .head() – first N observations
- .tail() – last N observations
- .describe() – statistics of the quantitative data
- .dtypes – the data types of the columns
- .columns – names of the columns
- .shape – the # of (rows, columns)

Useful PANDAS functions

- .read_csv() # loa

Accessing/processi

- df["column_nar
  - .max(), .min
  - .idxmax(), .
- <dataframe> <
- .loc[] – label-ba
- .iloc[] – index-b
- .sort_values()
- .isnull(), .notnul
- .dropna()
- .any()
- .values() E.g., df[
  - (df['name'] ==
- [0:3] *# grab the DataFrame*

Grouping/Splitting/Aggregating:

- groupby()

```
>>> df
      name         toy       born
0    Alfred         NaN        NaT
1    Batman   Batmobile 1940-04-25
2  Catwoman    Bullwhip        NaT
```

Drop the rows where at least one element is missing.

```
>>> df.dropna()
     name         toy       born
1  Batman   Batmobile 1940-04-25
```

Drop the columns where at least one element is missing.

```
>>> df.dropna(axis='columns')
      name
0    Alfred
1    Batman
2  Catwoman
```

ata

Useful PANDAS functions

- .read_csv() # loads a .csv file

Accessing/processing:

- df["column_name"]
  - .max(), .min(),
  - .idxmax(), .idxmin()
- <dataframe> <
- .loc[] – label-ba
- .iloc[] – index-b
- .sort_values()
- .isnull(), .notnul
- .dropna()
- .any()
- .values() E.g.,
  - (df['name'] ==
- [0:3] *# grab the DataFrame*

Grouping/Splitting/Aggregating:

- .groupby(),
- .get_groups()
- .drop()
- .merge(), .concat() .aggregate()
- .append()

"Do any of my values equal True?"

DataFrame → Series

Series → Boolean

Thank You!